



IT Architecture - Guidelines - Authentication and Authorization with Microsoft Entra ID

Summary

Concepts and Model with Microsoft Entra

Authentication (AuthN)

Authorization (AuthZ)

App Registrations and Service Principals

Multi-Domain Architecture

Overview

Strategy A: One API App Registration per Domain

Strategy B: One Shared API App Registration at the Gateway

Relationship Between UI Apps, Service Principals, and the API App Registration

Call Patterns: OBO, access_as_user, and Service-to-Service

Detailed Call Examples

Comparison: Per-Domain API Registration vs. Single Gateway API Registration

Pros and Cons Summary

Architectural Decision

Miscellaneous - API Gateway Integration

KrakenD

FAQs - Frequently Asked Questions

Summary

This document defines the authentication (AuthN) and authorization (AuthZ) architecture for a multi-domain platform built on Microsoft Entra ID (formerly Azure AD). The platform comprises multiple UI applications (SPAs), backend microservices organized into business domains, integration layers, and service principals within a single Microsoft Entra tenant.

The architecture must support:

- **Users** signing into UI applications via MSAL.js (or any Microsoft Entra authentication library) and accessing APIs across multiple domains.
- **Services and integrations** (microservices, daemons, managed identities) authenticating as themselves to call APIs.
- **Granular role-based access control (RBAC)** using custom app roles, applicable to both users and services.
- **Cross-domain calls**, where a service in one domain calls an API in another domain, with or without user context.

Two primary API registration strategies are evaluated — one API app registration per domain versus one shared API app registration at the gateway level — and a final architectural

recommendation is provided.

Concepts and Model with Microsoft Entra

Authentication (AuthN)

Authentication in Microsoft Entra is based on the [OAuth 2.0 and OpenID Connect protocols](#).

Two authentication patterns exist:

- **User authentication (delegated):** A user signs into a UI application via an interactive flow (Authorization Code + PKCE for SPAs). The user's identity is represented in an ID token and access tokens.
- **Application authentication (app-only):** A service authenticates as itself using its own credentials (client secret, certificate, or managed identity) via the [client credentials flow](#). No user is involved.

Authorization (AuthZ)

Authorization determines what an authenticated caller can do. Microsoft Entra provides two mechanisms that map to the two authentication patterns:

Mechanism	Token claim	Applies to	Defined via	Purpose
Delegated permissions (scopes)	scp	User + app combination	"Expose an API" → Scopes on the API app registration	Controls what the app is allowed to request on behalf of the user
Application permissions (app roles)	roles	Users, groups, or service principals	"App roles" on the API app registration	Controls what the caller is authorized to be

In a delegated flow, [the effective access is the intersection](#) of the delegated scope (what the app can do) and the user's own permissions (what the user can do, expressed via roles). In an app-only flow, the effective access equals the app roles assigned to the service principal — there is no user to intersect with.

App Registrations and Service Principals

Every application in Microsoft Entra has two objects ([Microsoft Learn](#)):

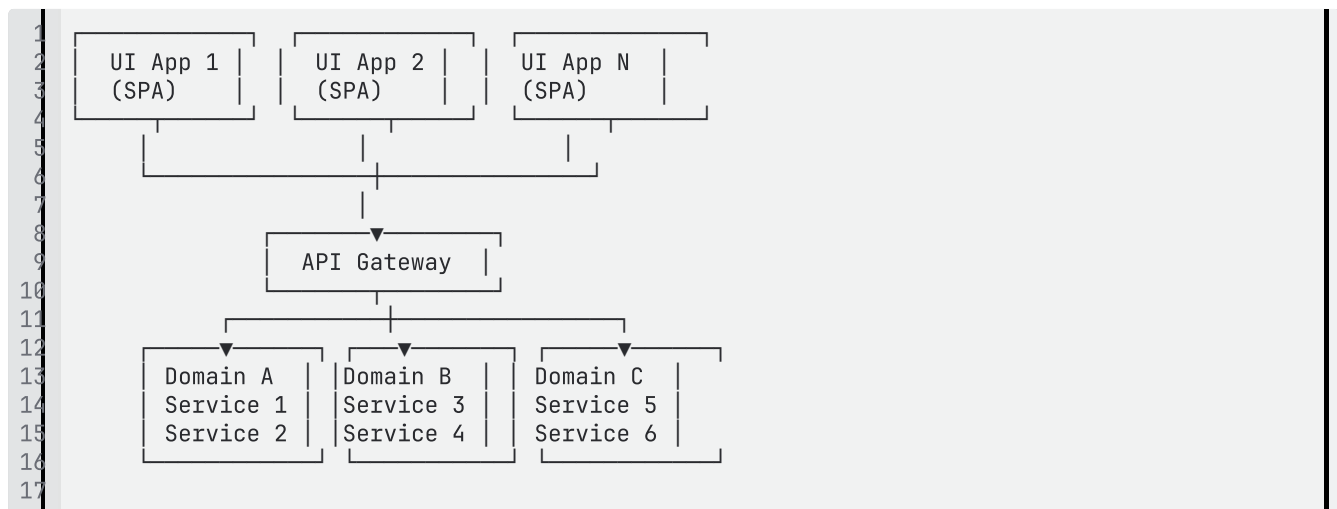
- **Application object (app registration):** The global definition/template — defines the app's identity (client ID / app ID), redirect URIs, scopes, app roles, and credentials.
- **Service principal (enterprise application):** The tenant-local instance — created automatically in the home tenant. This is where users, groups, and other service principals are assigned roles.

Roles are always defined on the app registration but assigned on the enterprise application (service principal).

Multi-Domain Architecture

Overview

The platform is composed of multiple business domains, each containing one or more microservices exposing APIs. Multiple UI applications (SPAs) consume APIs across these domains. An API gateway sits between clients and backend services.



There are also non-UI clients:

- **Integration layers** (service principals with client credentials)
- **Other microservices** calling across domains (using OBO or client credentials)
- **Managed identities** on Azure-hosted resources

Strategy A: One API App Registration per Domain

Each business domain gets its own app registration with its own application ID URI, scopes, and app roles.

```

1 App Registration: "DomainA-API"
2   | Application ID URI: api://domain-a
3   | Scopes: api://domain-a/read, api://domain-a/write
  
```

```

4   └─ App Roles:
5       └─ DomainA.Reader   (Allowed: Users/Groups + Applications)
6       └─ DomainA.Writer   (Allowed: Users/Groups + Applications)
7       └─ DomainA.Admin    (Allowed: Users/Groups + Applications)
8
9   App Registration: "DomainB-API"
10  └─ Application ID URI: api://domain-b
11  └─ Scopes: api://domain-b/read, api://domain-b/write
12  └─ App Roles:
13      └─ DomainB.Reader    (Allowed: Users/Groups + Applications)
14      └─ DomainB.Operator  (Allowed: Users/Groups + Applications)
15

```

Characteristics:

- Each domain is an independent audience (`aud`) in access tokens.
- Clients must acquire **separate tokens** per domain — one token per audience.
- Roles are domain-scoped: `DomainA.Reader` is meaningfully different from `DomainB.Reader`.
- Each domain team independently manages its own roles and scopes.
- Cross-domain calls require OBO (On-Behalf-Of) or separate client credentials tokens.

Strategy B: One Shared API App Registration at the Gateway

A single app registration represents the entire API surface, typically associated with the API gateway.

```

1   App Registration: "Platform-API"
2   └─ Application ID URI: api://platform-api
3   └─ Scopes: api://platform-api/read, api://platform-api/write
4   └─ App Roles:
5       └─ DomainA.Reader
6       └─ DomainA.Writer
7       └─ DomainA.Admin
8       └─ DomainB.Reader
9       └─ DomainB.Operator
10      └─ ... (all domains' roles in one place)
11

```

Characteristics:

- All backend services share one audience (`aud`).
 - Clients acquire a **single token** that works across all domains.
 - All roles are defined in one app registration, regardless of which domain they belong to.
 - The gateway validates the token; each backend service checks specific roles from the same token.
 - Simpler token management for UI apps — one `acquireTokenSilent` call.
-

Relationship Between UI Apps, Service Principals, and the API App Registration

How UI Apps Relate to the API

UI applications are registered as **client apps** in Entra. They do not define their own roles or scopes — they are consumers. Their relationship to the API app registration is established through:

1. **API Permissions (delegated):** The UI app registration declares which scopes it needs from the API (e.g., `api://domain-a/read`). These appear under "API Permissions → Delegated" in the Entra portal.
2. `knownClientApplications`: The API app registration's manifest can list the UI app's client ID in `knownClientApplications`, enabling [bundled consent](#) — the user consents to both the UI's permissions and the API's permissions in a single prompt.
3. **Role assignment:** Users are assigned roles on the **API's enterprise application**, not on the UI app's enterprise application. The UI app simply acquires a token scoped to the API; the roles claim comes from the user's assignment on the API's service principal.

```
1 UI App Registration (MyUI)                                API App Registration (DomainA-API)
2 |— Client ID: aaa-bbb                                     |— Client ID: xxx-yyy
3 |— Platform: SPA                                          |— App ID URI: api://domain-a
4 |— API Permissions:                                       |— Scopes: read, write
5 |   |— DomainA-API (Delegated): read, write              |— App Roles: DomainA.Reader, ...
6 |— No app roles                                           |— knownClientApplications: ["aaa-bbb"]
7
8                                                         Enterprise App (DomainA-API):
9 |— User "Vincenzo Verrone" → DomainA.Reader
10 |— Group "Admins" → DomainA.Admin
11 |— SP "Integration-X" → DomainA.Reader
12
```

In the above example, when Vincenzo signs in to MyUI and requests a token for `api://domain-a/read`:

```
1 {
2   "aud": "api://domain-a",
3   "sub": "vincenzo-object-id",
4   "scp": "read",
5   "roles": ["DomainA.Reader"],
6   "azp": "aaa-bbb"
7 }
8
```

- `scp: "read"` — the UI app requested (and was granted) the `read` delegated scope.
- `roles: ["DomainA.Reader"]` — Vincenzo was assigned this role on DomainA-API's enterprise app.
- `azp: "aaa-bbb"` — identifies the calling client app (the UI).

How Service Principals Relate to the API

Services (microservices, daemons, managed identities) relate to the API app registration through **application permissions**:

1. **API Permissions (application)**: The service's app registration declares which app roles it needs from the API (e.g., `DomainA.Reader`). These appear under "API Permissions → Application" in the Entra portal.
2. **Admin consent**: An admin grants these permissions, creating an [appRoleAssignment](#) between the service's service principal and the API's service principal.
3. **Token request**: The service uses client credentials with `scope=api://domain-a/.default`. The resulting token contains all admin-consented app roles in the `roles` claim.

```
1 {  
2   "aud": "api://domain-a",  
3   "roles": ["DomainA.Reader"],  
4   "azp": "integration-service-client-id",  
5   "sub": "integration-service-object-id"  
6 }  
7
```

No `scp` claim exists — there is no user context.

Call Patterns: OBO, access_as_user, and Service-to-Service

Pattern 1: UI-to-Service Calls (Delegated)

The user signs into the UI, which acquires a token for the target API and calls it directly (or through the gateway).

```
1 User → UI App (MSAL.js) → acquireTokenSilent(scopes: ["api://domain-a/read"])  
2                               → HTTP GET /api/data  
3                               Authorization: Bearer <token>  
4                               → DomainA-Service validates token  
5                               checks: aud == api://domain-a  
6                               scp contains "read"  
7                               roles contains required role  
8
```

Flow: Authorization Code + PKCE

Token claims: `scp` + `roles`

User context: Yes — the API knows who the user is.

Pattern 2: Service-to-Service Calls (Client Credentials)

A backend service or daemon authenticates as itself and calls another API. No user is involved.

```
1 Integration-Service-X → POST /oauth2/v2.0/token  
2                       grant_type=client_credentials
```

```

3         client_id=<service-client-id>
4         client_secret=<secret>
5         scope=api://domain-a/.default
6     → HTTP GET /api/data
7         Authorization: Bearer <token>
8     → DomainA-Service validates token
9         checks: aud == api://domain-a
10                roles contains required role
11                (no scp – app-only token)
12

```

Flow: Client Credentials

Token claims: `roles` only (no `scp`)

User context: None — the API sees only the service identity.

This pattern applies to:

- Microservices calling other microservices without user context
- Daemons, scheduled jobs, background workers
- Managed identities on Azure resources (using `DefaultAzureCredential` instead of client secret)

Pattern 3: On-Behalf-Of (OBO) — Propagating User Context Across Services

When a service receives a user's delegated token and needs to call another API while preserving the user's identity, it uses the [On-Behalf-Of flow](#).

```

1 User → UI App → Token for DomainA-API (delegated)
2     → DomainA-Service receives token
3     → DomainA-Service needs data from DomainB-API
4     → POST /oauth2/v2.0/token
5         grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer
6         client_id=<domainA-service-client-id>
7         client_secret=<domainA-service-secret>
8         assertion=<original-user-token>
9         scope=api://domain-b/read
10        requested_token_use=on_behalf_of
11    → Receives NEW token for DomainB-API, still representing the user
12    → HTTP GET /domain-b/api/data
13        Authorization: Bearer <new-obo-token>
14

```

Key characteristics of OBO:

- The new token has `aud: api://domain-b` — a different audience.
- The user's identity is preserved: `sub` , `name` , etc. remain the user's claims.
- The OBO flow uses **delegated scopes only**, not application roles. As [Microsoft documents](#): "It only uses delegated scopes and not application roles. Roles remain attached to the principal (the user) and never to the application operating on the user's behalf."
- The user's `roles` for DomainB-API are still included in the new token (based on the user's assignments on DomainB-API's enterprise app).

- DomainA-Service's app registration must have delegated API permissions for `api://domain-b/read`.

The `access_as_user` Scope

The `access_as_user` scope is a common convention (not a system-reserved name) used to represent the most basic delegated permission an API exposes: "this app can access the API as the signed-in user." It is defined under "Expose an API" on the API app registration.

```
1 API App Registration: DomainA-API
2   └─ Expose an API → Scopes:
3     └─ api://domain-a/access_as_user
4         Description: "Access DomainA API as the signed-in user"
5         Admin consent required: No
6
```

When a UI app or a middle-tier service (via OBO) requests `api://domain-a/access_as_user`, the resulting token's `scp` claim contains `access_as_user`. The API then uses the `roles` claim to determine what the user can actually do.

This pattern separates concerns:

- **Scope (`access_as_user`):** "The app is allowed to call this API on behalf of the user" — a coarse gatekeeper.
- **Roles (`DomainA.Reader`, etc.):** "The user has this level of access" — fine-grained authorization.

Using `access_as_user` is simpler than defining granular scopes like `read`, `write`, `admin` when app roles already handle fine-grained authorization. The scope serves primarily as a consent and audience-targeting mechanism, while roles do the real authorization work.

Detailed Call Examples

Example 1: UI User Calling a Single Domain

Vincenzo opens UI App 1 and navigates to a page that loads data from Domain A.

```
1 // UI App 1 – MSAL.js
2 const tokenResponse = await msalInstance.acquireTokenSilent({
3   scopes: ["api://domain-a/access_as_user"],
4   account: msalInstance.getActiveAccount()
5 });
6
7 const response = await fetch("https://gateway.example.com/domain-a/data", {
8   headers: { Authorization: `Bearer ${tokenResponse.accessToken}` }
9 });
10
```

Token contents:


```

1  {
2    "aud": "api://domain-a",
3    "iss": "https://login.microsoftonline.com/{tenant-id}/v2.0",
4    "sub": "vincenzo-oid",
5    "name": "Vincenzo Verrone",
6    "scp": "access_as_user",
7    "roles": ["DomainA.Reader"],
8    "azp": "ui-app-1-client-id"
9  }
10

```

API validation (DomainA-Service):

```

1  [Authorize(Roles = "DomainA.Reader,DomainA.Admin")]
2  [RequiredScope("access_as_user")]
3  [HttpGet("data")]
4  public IActionResult GetData() => Ok(data);
5

```

Example 2: UI User Calling Multiple Domains (Single Token Cannot Cover Multiple Audiences)

Vincenzo's page also needs data from Domain B. Because [an access token is bound to a single audience](#), the UI must acquire a **separate token** for Domain B.

```

1  // Token for Domain A
2  const tokenA = await msalInstance.acquireTokenSilent({
3    scopes: ["api://domain-a/access_as_user"],
4    account: msalInstance.getActiveAccount()
5  });
6
7  // Token for Domain B – separate request, same session, NO re-login
8  const tokenB = await msalInstance.acquireTokenSilent({
9    scopes: ["api://domain-b/access_as_user"],
10   account: msalInstance.getActiveAccount()
11  });
12
13 // Call Domain A
14 await fetch("https://gateway.example.com/domain-a/data", {
15   headers: { Authorization: `Bearer ${tokenA.accessToken}` }
16 });
17
18 // Call Domain B
19 await fetch("https://gateway.example.com/domain-b/data", {
20   headers: { Authorization: `Bearer ${tokenB.accessToken}` }
21 });
22

```

The user does not sign in again — MSAL uses the cached refresh token to silently obtain the second token. However, the UI must manage multiple tokens and attach the correct one per API call.

With Strategy B (single API app registration), this simplifies to:

```

1  // Single token for the entire platform
2  const token = await msalInstance.acquireTokenSilent({
3    scopes: ["api://platform-api/access_as_user"],
4    account: msalInstance.getActiveAccount()
5  });
6
7  // Same token for all domain calls

```

```

8  await fetch("https://gateway.example.com/domain-a/data", {
9    headers: { Authorization: `Bearer ${token.accessToken}` }
10 });
11
12 await fetch("https://gateway.example.com/domain-b/data", {
13   headers: { Authorization: `Bearer ${token.accessToken}` }
14 });
15

```

One token, one audience (`api://platform-api`), all domain roles included.

Example 3: Service Principal Calling an API (Client Credentials)

An integration service (e.g., a nightly data sync job) calls Domain A with no user context.

```

1  # 1. Acquire token
2  curl -X POST https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token \
3    -d "grant_type=client_credentials" \
4    -d "client_id={integration-service-client-id}" \
5    -d "client_secret={secret}" \
6    -d "scope=api://domain-a/.default"
7
8  # 2. Call API with the token
9  curl -H "Authorization: Bearer {access_token}" \
10     https://gateway.example.com/domain-a/data
11

```

Token contents:

```

1  {
2    "aud": "api://domain-a",
3    "roles": ["DomainA.Reader"],
4    "azp": "integration-service-client-id",
5    "sub": "integration-service-sp-oid"
6  }
7

```

No `scp` claim. No user identity. The API checks `roles` only.

For **managed identities**, the code uses `DefaultAzureCredential` instead of explicit secrets:

```

1  var credential = new DefaultAzureCredential();
2  var token = await credential.GetTokenAsync(
3    new TokenRequestContext(new[] { "api://domain-a/.default" }));
4

```

The managed identity must have the app role assigned on the API's enterprise application, just like any other service principal.

Example 4: Service-to-Service with OBO (Propagating User Context)

DomainA-Service receives a user's delegated token and needs to call DomainB-API to complete the request — while preserving the user's identity.

```

1  // DomainA-Service - .NET with Microsoft.Identity.Web
2  [Authorize]
3  [HttpGet("composite-data")]

```

```

4 public async Task<IActionResult> GetCompositeData()
5 {
6     // The incoming token is for api://domain-a, from the UI on behalf of Vincenzo
7
8     // Use OBO to get a token for DomainB, still as Vincenzo
9     var tokenResult = await _tokenAcquisition.GetAccessTokenForUserAsync(
10         new[] { "api://domain-b/access_as_user" });
11
12     // Call DomainB with the OBO token
13     _httpClient.DefaultRequestHeaders.Authorization =
14         new AuthenticationHeaderValue("Bearer", tokenResult);
15
16     var domainBData = await _httpClient.GetAsync("https://domain-b-service/api/data");
17
18     // Combine and return
19     return Ok(new { domainAData = localData, domainBData });
20 }
21

```

OBO token contents (for DomainB):

```

1 {
2   "aud": "api://domain-b",
3   "sub": "vincenzo-oid",
4   "name": "Vincenzo Verrone",
5   "scp": "access_as_user",
6   "roles": ["DomainB.Reader"],
7   "azp": "domain-a-service-client-id"
8 }
9

```

The user's identity is preserved. DomainB sees Vincenzo as the caller, not DomainA-Service.

Example 5: Service-to-Service without User Context (Client Credentials, Cross-Domain)

DomainA-Service needs reference data from DomainB-API during a background process — no user involved.

```

1 // DomainA-Service - client credentials to DomainB
2 var app = ConfidentialClientApplicationBuilder
3     .Create(config.DomainAServiceClientId)
4     .WithClientSecret(config.DomainAServiceSecret)
5     .WithAuthority($"https://login.microsoftonline.com/{config.TenantId}")
6     .Build();
7
8 var result = await app.AcquireTokenForClient(
9     new[] { "api://domain-b/.default" })
10    .ExecuteAsync();
11
12 _httpClient.DefaultRequestHeaders.Authorization =
13     new AuthenticationHeaderValue("Bearer", result.AccessToken);
14
15 var refData = await _httpClient.GetAsync("https://domain-b-service/api/reference");
16

```

Token: app-only, roles only, aud: api://domain-b, caller is DomainA-Service's identity.

Comparison: Per-Domain API Registration vs. Single Gateway API Registration

Aspect	Strategy A: One API per Domain	Strategy B: One Shared API at Gateway
Token audience	One <code>aud</code> per domain (<code>api://domain-a</code> , <code>api://domain-b</code> , ...)	Single <code>aud</code> for all (<code>api://platform-api</code>)
Tokens per UI session	Multiple — one per domain the UI calls	One — covers all domains
UI complexity	Higher — must manage multiple tokens and map them to API calls	Lower — single token for all calls
Role isolation	Strong — each domain's roles are independent. A breach in Domain A's app registration does not affect Domain B's roles	Weak — all roles in one registration. Compromise of the single app registration exposes all domain roles
Team autonomy	Each domain team owns its app registration and manages roles/scopes independently	Central team manages the single app registration. Domain teams must coordinate role changes
Cross-domain calls	Requires OBO (delegated) or separate client credentials token (app-only) for each target domain	Backend services share the same audience — no OBO needed if the gateway token is forwarded. However, this weakens audience validation at the backend level
Token size	Smaller — contains only roles for one domain	Larger — contains all roles across all domains. May hit token size limits with many roles
Audience validation	Strict — each backend service validates that <code>aud</code> matches its own API	Weaker — all services accept the same <code>aud</code> . A token for "the platform" can be forwarded to any service
Blast radius of a leaked token	Limited to one domain's APIs	All APIs across all domains
Consent experience	Multiple consent prompts possible (mitigated by	Single consent prompt covers everything

	knownClientApplications and admin consent)	
Entra limits	Each app registration independently within Entra limits	Single app registration bears all roles and permissions — risk of hitting the 1200-entry manifest limit with many domains
OBO flow	Natural fit — OBO exchanges tokens across audiences. User context propagates cleanly	Not needed for same-audience calls — but also means no audience boundary enforcement between services
Microservice architecture alignment	Aligns with bounded contexts / domain-driven design	Aligns with monolithic gateway / BFF pattern
Scalability	Scales well as domains are added — each is independent	Adding domains means more roles in one registration — becomes harder to manage
Compliance/audit	Per-domain token logs clearly show which domain was accessed	Token logs show access to "the platform" — harder to determine which domain was targeted without application-level logging

Pros and Cons Summary

Strategy A: One API App Registration per Domain

Pros:

- Strong security boundaries: audience isolation, domain-scoped blast radius.
- Domain team autonomy: each team manages its own roles and scopes.
- Aligned with microservice and DDD principles (bounded contexts).
- Smaller tokens, no risk of hitting manifest limits.
- Clean OBO flow for cross-domain calls with user context.
- Better auditability: token audience directly identifies the target domain.

Cons:

- UI apps must acquire and manage multiple tokens (one per domain).
- Slightly more complex client-side code (MSAL token mapping per API call).
- More app registrations to manage overall.
- Cross-domain service calls require explicit OBO or client credentials token acquisition.

Strategy B: One Shared API App Registration at Gateway

Pros:

- Simplest UI experience: one token for all API calls.
- No OBO needed for intra-platform calls (same audience).
- Single consent prompt for the entire platform.
- Fewer app registrations to manage.

Cons:

- No audience-level isolation between domains — a token valid for Domain A is also valid for Domain B.
- All roles centralized in one manifest — coordination bottleneck, risk of manifest size limits.
- Larger token size with many roles.
- Leaked token exposes all domains.
- Weaker compliance posture: logs don't distinguish target domains by audience.
- Tighter coupling: domain teams cannot independently evolve their authorization model.

Architectural Decision

Recommendation: Strategy A — One API App Registration per Domain

For a multi-domain architecture within a single tenant, the **per-domain API registration** strategy (Strategy A) is the recommended approach. This recommendation is driven by:

1. **Security-first design.** Audience isolation (aud per domain) ensures that a token obtained for Domain A cannot be replayed against Domain B. This is a fundamental security boundary that Strategy B sacrifices for convenience.
2. **Team scalability.** As the platform grows and domain teams evolve independently, each team owns its app registration, roles, and scopes. There is no central bottleneck or coordination required when adding new roles.
3. **Bounded context alignment.** The per-domain model maps naturally to Domain-Driven Design. Each domain's authorization model is self-contained and can evolve without affecting other domains.

4. **Manageable UI complexity.** While UI apps must manage multiple tokens, MSAL.js handles this transparently — `acquireTokenSilent` caches tokens per resource and the user never signs in more than once. The implementation cost is a mapping layer that attaches the correct token per API call, which is straightforward with a wrapper:

```
1 // Token management abstraction
2 const apiTokenMap = {
3   "domain-a": "api://domain-a/access_as_user",
4   "domain-b": "api://domain-b/access_as_user",
5 };
6
7 async function callApi(domain, path) {
8   const token = await msalInstance.acquireTokenSilent({
9     scopes: [apiTokenMap[domain]],
10    account: msalInstance.getActiveAccount()
11  });
12  return fetch(`https://gateway.example.com/${domain}/${path}`, {
13    headers: { Authorization: `Bearer ${token.accessToken}` }
14  });
15 }
16
```

5. **Clean cross-domain communication.** OBO flow for user-context propagation and client credentials for service-to-service calls are well-established patterns. They add explicit intent to each cross-domain call, making the system easier to reason about and audit.

Mitigations for Strategy A's Complexity

- `knownClientApplications`: Add all UI app client IDs to each API app registration's manifest to enable [bundled consent](#), reducing consent prompts to a single interaction.
- **Admin consent:** For enterprise single-tenant deployments, pre-consent all permissions via admin consent — users never see consent prompts at all.
- **Token caching:** MSAL's built-in cache ensures that multiple `acquireTokenSilent` calls do not result in multiple network requests unless tokens are expired.
- **Gateway-level validation:** The API gateway validates token signature and basic claims (issuer, expiration). Backend services validate audience and roles — this layered approach prevents invalid tokens from reaching backends.

When to Consider Strategy B

Strategy B (single gateway API registration) may be appropriate when:

- The platform has very few domains (2-3) that are unlikely to diverge.
- Domain teams are small or the same team manages all domains.
- Simplifying the UI is the overriding priority and security boundaries between domains are not critical.

- The API gateway acts as a true BFF (Backend-for-Frontend) that aggregates and re-authorizes all calls internally, and backend services are never exposed directly.

Even in these cases, transitioning to Strategy A later becomes increasingly difficult as the platform grows, making it preferable to start with per-domain registrations from the outset.

Miscellaneous - API Gateway Integration

KrakenD

KrakenD is a stateless, high-performance API gateway that validates JWT tokens without token introspection — it verifies the token signature locally using the public keys from the identity provider's JWK endpoint. This section covers how to configure KrakenD as the gateway layer for both Strategy A (per-domain) and Strategy B (single API), including token validation, role/scope enforcement, and claim propagation to backend services.

How KrakenD Fits in the Architecture

KrakenD sits between the clients (UI apps, service principals) and the backend services. Its role in the AuthN/AuthZ flow is:

1. **Validate the JWT signature** against Microsoft Entra's public keys (via [JWK URL](#)).
2. **Check token claims** — audience, issuer, expiration, roles, and scopes.
3. **Reject unauthorized requests** before they reach backend services.
4. **Propagate claims** (e.g., user identity, roles) as HTTP headers to backends.
5. Optionally, **acquire tokens for backend calls** using client credentials (`auth/client-credentials`).

KrakenD does not generate tokens and does not call back to the identity provider for introspection — all validation is offline based on the JWK keys, which are [cached in memory](#).

```
1 Client → KrakenD (validates JWT, checks roles/scopes)
2       | passes request + propagated claim headers
3       ▼
4       Backend Service (validates audience, fine-grained authorization)
5
```

Microsoft Entra JWK URL

For a single-tenant deployment, the JWK URL for Microsoft Entra v2.0 is:

```
1 https://login.microsoftonline.com/{tenant-id}/discovery/v2.0/keys
2
```

The issuer (`iss`) in tokens will be:


```
1 https://login.microsoftonline.com/{tenant-id}/v2.0
2
```

Replace `{tenant-id}` with the actual tenant GUID.

Configuration: Strategy A (Per-Domain API Registration)

With per-domain registrations, each endpoint group validates tokens against a different audience. KrakenD's `auth/validator` is configured per endpoint.

Full `krakend.json` Example

```
1 {
2   "version": 3,
3   "$schema": "https://www.krakend.io/schema/krakend.json",
4   "extra_config": {
5     "auth/validator": {
6       "shared_cache_duration": 900
7     }
8   },
9   "endpoints": [
10    {
11      "@comment": "Domain A - read data",
12      "endpoint": "/domain-a/data",
13      "method": "GET",
14      "input_headers": ["Authorization", "X-User-Id", "X-User-Roles"],
15      "extra_config": {
16        "auth/validator": {
17          "alg": "RS256",
18          "jwk_url": "https://login.microsoftonline.com/{tenant-id}/discovery/v2.0/keys",
19          "cache": true,
20          "issuer": "https://login.microsoftonline.com/{tenant-id}/v2.0",
21          "audience": ["api://domain-a"],
22          "roles_key": "roles",
23          "roles": ["DomainA.Reader", "DomainA.Admin"],
24          "scopes_key": "scp",
25          "scopes_matcher": "any",
26          "scopes": ["access_as_user"],
27          "propagate_claims": [
28            ["sub", "X-User-Id"],
29            ["roles", "X-User-Roles"]
30          ],
31          "operation_debug": false
32        }
33      },
34      "backend": [
35        {
36          "url_pattern": "/api/data",
37          "host": ["http://domain-a-service:8080"]
38        }
39      ]
40    },
41    {
42      "@comment": "Domain A - write data",
43      "endpoint": "/domain-a/data",
44      "method": "POST",
45      "input_headers": ["Authorization", "Content-Type", "X-User-Id", "X-User-Roles"],
46      "extra_config": {
47        "auth/validator": {
48          "alg": "RS256",
49          "jwk_url": "https://login.microsoftonline.com/{tenant-id}/discovery/v2.0/keys",
50          "cache": true,
51          "issuer": "https://login.microsoftonline.com/{tenant-id}/v2.0",
```

```

52         "audience": ["api://domain-a"],
53         "roles_key": "roles",
54         "roles": ["DomainA.Writer", "DomainA.Admin"],
55         "propagate_claims": [
56             ["sub", "X-User-Id"],
57             ["roles", "X-User-Roles"]
58         ]
59     },
60 },
61 "backend": [
62     {
63         "url_pattern": "/api/data",
64         "host": ["http://domain-a-service:8080"]
65     }
66 ]
67 },
68 {
69     "@comment": "Domain B – read data",
70     "endpoint": "/domain-b/data",
71     "method": "GET",
72     "input_headers": ["Authorization", "X-User-Id", "X-User-Roles"],
73     "extra_config": {
74         "auth/validator": {
75             "alg": "RS256",
76             "jwk_url": "https://login.microsoftonline.com/{tenant-id}/discovery/v2.0/keys",
77             "cache": true,
78             "issuer": "https://login.microsoftonline.com/{tenant-id}/v2.0",
79             "audience": ["api://domain-b"],
80             "roles_key": "roles",
81             "roles": ["DomainB.Reader", "DomainB.Operator"],
82             "scopes_key": "scp",
83             "scopes_matcher": "any",
84             "scopes": ["access_as_user"],
85             "propagate_claims": [
86                 ["sub", "X-User-Id"],
87                 ["roles", "X-User-Roles"]
88             ]
89         }
90     },
91     "backend": [
92         {
93             "url_pattern": "/api/data",
94             "host": ["http://domain-b-service:8080"]
95         }
96     ]
97 }
98 ]
99 }
100

```

Key points for Strategy A:

- Each endpoint group has a **different** audience matching the domain's application ID URI (api://domain-a, api://domain-b).
- KrakenD enforces **roles at the gateway level** — only tokens with the required roles pass through.
- The scopes_key: "scp" and scopes fields validate delegated scopes for user tokens. For app-only tokens (client credentials), no scp claim exists — KrakenD will skip scope validation if the claim is absent, and roles validation still applies.

- `propagate_claims` sends the `sub` (user/service identity) and `roles` as custom headers to backends, so backends can do fine-grained authorization without re-parsing the JWT.
- The `input_headers` list must include the propagated header names (`X-User-Id` , `X-User-Roles`), otherwise KrakenD's zero-trust policy blocks them from reaching backends.
- `shared_cache_duration: 900` at the top level caches Entra's JWK keys for 15 minutes across all endpoints, avoiding redundant key fetches.

Handling Both Delegated and App-Only Tokens

A challenge with Strategy A is that the same endpoint may receive both user tokens (with `scp`) and service tokens (with only `roles` , no `scp`). If you set `scopes` in the validator, KrakenD will reject app-only tokens because they lack the `scp` claim.

Solution: For endpoints that accept both token types, omit `scopes_key` and `scopes` from the KrakenD validator and let the backend handle scope checking for delegated tokens. KrakenD validates roles (which both token types have); the backend distinguishes between delegated and app-only by checking for the presence of `scp` .

```

1  {
2    "@comment": "Endpoint accepting both delegated and app-only tokens",
3    "endpoint": "/domain-a/data",
4    "method": "GET",
5    "input_headers": ["Authorization", "X-User-Id", "X-User-Roles", "X-Token-Scp"],
6    "extra_config": {
7      "auth/validator": {
8        "alg": "RS256",
9        "jwk_url": "https://login.microsoftonline.com/{tenant-id}/discovery/v2.0/keys",
10       "cache": true,
11       "issuer": "https://login.microsoftonline.com/{tenant-id}/v2.0",
12       "audience": ["api://domain-a"],
13       "roles_key": "roles",
14       "roles": ["DomainA.Reader", "DomainA.Admin"],
15       "propagate_claims": [
16         ["sub", "X-User-Id"],
17         ["roles", "X-User-Roles"],
18         ["scp", "X-Token-Scp"]
19       ]
20     }
21   },
22   "backend": [
23     {
24       "url_pattern": "/api/data",
25       "host": ["http://domain-a-service:8080"]
26     }
27   ]
28 }
29

```

The backend receives `X-Token-Scp` (present for delegated tokens, absent for app-only) and can branch logic accordingly.

Configuration: Strategy B (Single Gateway API Registration)

With a single API registration, all endpoints share the same audience. The configuration is simpler — a single `audience` value applies everywhere.

```
1 {
2   "version": 3,
3   "$schema": "https://www.krakend.io/schema/krakend.json",
4   "extra_config": {
5     "auth/validator": {
6       "shared_cache_duration": 900
7     }
8   },
9   "endpoints": [
10    {
11      "endpoint": "/domain-a/data",
12      "method": "GET",
13      "input_headers": ["Authorization", "X-User-Id", "X-User-Roles"],
14      "extra_config": {
15        "auth/validator": {
16          "alg": "RS256",
17          "jwk_url": "https://login.microsoftonline.com/{tenant-id}/discovery/v2.0/keys",
18          "cache": true,
19          "issuer": "https://login.microsoftonline.com/{tenant-id}/v2.0",
20          "audience": ["api://platform-api"],
21          "roles_key": "roles",
22          "roles": ["DomainA.Reader", "DomainA.Admin"],
23          "propagate_claims": [
24            ["sub", "X-User-Id"],
25            ["roles", "X-User-Roles"]
26          ]
27        }
28      },
29      "backend": [
30        {
31          "url_pattern": "/api/data",
32          "host": ["http://domain-a-service:8080"]
33        }
34      ]
35    },
36    {
37      "endpoint": "/domain-b/data",
38      "method": "GET",
39      "input_headers": ["Authorization", "X-User-Id", "X-User-Roles"],
40      "extra_config": {
41        "auth/validator": {
42          "alg": "RS256",
43          "jwk_url": "https://login.microsoftonline.com/{tenant-id}/discovery/v2.0/keys",
44          "cache": true,
45          "issuer": "https://login.microsoftonline.com/{tenant-id}/v2.0",
46          "audience": ["api://platform-api"],
47          "roles_key": "roles",
48          "roles": ["DomainB.Reader", "DomainB.Operator"],
49          "propagate_claims": [
50            ["sub", "X-User-Id"],
51            ["roles", "X-User-Roles"]
52          ]
53        }
54      },
55      "backend": [
56        {
57          "url_pattern": "/api/data",
58          "host": ["http://domain-b-service:8080"]
59        }
60      ]
61    }
62  ]
63 }
```

```

61     }
62   ]
63 }
64

```

Key difference from Strategy A: The `audience` is the same across all endpoints (`api://platform-api`). Role-based access control per domain is enforced through different `roles` arrays per endpoint — KrakenD checks that the token contains at least one of the listed roles.

KrakenD Client Credentials for Backend-to-Backend

When KrakenD itself needs to call a backend that requires its own token (e.g., a protected microservice that doesn't trust the forwarded user token), configure `auth/client-credentials` on the backend:

```

1  {
2    "endpoint": "/domain-a/internal-data",
3    "method": "GET",
4    "extra_config": {
5      "auth/validator": {
6        "alg": "RS256",
7        "jwk_url": "https://login.microsoftonline.com/{tenant-id}/discovery/v2.0/keys",
8        "cache": true,
9        "audience": ["api://domain-a"]
10     }
11   },
12   "backend": [
13     {
14       "url_pattern": "/api/internal-data",
15       "host": ["http://domain-b-service:8080"],
16       "extra_config": {
17         "auth/client-credentials": {
18           "client_id": "krakend-service-principal-client-id",
19           "client_secret": "krakend-service-principal-secret",
20           "token_url": "https://login.microsoftonline.com/{tenant-id}/oauth2/v2.0/token",
21           "endpoint_params": {
22             "audience": ["api://domain-b"]
23           },
24           "scopes": "api://domain-b/.default"
25         }
26       }
27     }
28   ]
29 }
30

```

In this scenario, KrakenD validates the incoming user token for Domain A, then acquires its own client credentials token to call Domain B's backend. KrakenD handles token refresh automatically.

Reducing Configuration Repetition

The examples above repeat the `auth/validator` block in every endpoint. In production, use KrakenD's [flexible configuration](#) (Go templates) to extract shared blocks:

```

1  {{/* templates/entra_validator.tmpl */}}
2  "auth/validator": {
3    "alg": "RS256",
4    "jwk_url": "https://login.microsoftonline.com/{{ .tenant_id }}/discovery/v2.0/keys",
5    "cache": true,
6    "issuer": "https://login.microsoftonline.com/{{ .tenant_id }}/v2.0",
7    "audience": ["{{ .audience }}"],
8    "roles_key": "roles",
9    "roles": {{ .roles }},
10   "propagate_claims": [
11     ["sub", "X-User-Id"],
12     ["roles", "X-User-Roles"]
13   ]
14 }
15

```

Then reference it in endpoints:

```

1  "extra_config": {
2    {{ template "entra_validator.tmpl" (dict "tenant_id" .tenant_id "audience" "api://domain-a"
3  }}
4

```

Forwarding the Original Token vs. Propagating Claims

Two approaches exist for passing identity information to backends:

Approach	How	When to use
Forward Authorization on header	Add "Authorization" to input_headers	Backend needs the full JWT (e.g. for OBO flow to call downstream APIs)
Propagate claims as headers	Use propagate_claims to extract sub, roles, name, etc. into custom headers	Backend only needs identity metadata, not the raw token. Lighter and avoids backends re-parsing the JWT

For Strategy A with OBO, backends often need the original token to perform the OBO exchange, so forwarding Authorization is essential. For Strategy B, where no OBO is needed, propagating claims as headers is cleaner.

Summary: KrakenD Configuration by Strategy

Aspect	Strategy A (per-domain)	Strategy B (single API)
audience	Different per endpoint group (api://domain-a, api://domain-b)	Same for all endpoints (api://platform-api)

roles	Domain-specific per endpoint	Domain-specific per endpoint (same)
scopes validation	Optional — may skip for mixed delegated/app-only endpoints	Optional — same consideration
Token forwarding	Forward Authorization header for OBO	Propagate claims only (no OBO needed)
Client credentials (backend)	Needed when KrakenD calls cross-domain backends	Not needed (same audience)
Configuration repetition	Higher — different audience per endpoint group	Lower — same audience everywhere

FAQs - Frequently Asked Questions

"Why do I need 2 App IDs (one for the UI and one for the API)? Can't I just use one for both?"

Technically, yes — Entra allows a single app registration to serve as both the client (UI) and the resource (API). But it's a bad idea for anything beyond a quick prototype, for several reasons:

- **Security: separation of credentials.** The API app registration holds secrets or certificates (used for OBO, client credentials, downstream calls). A SPA should never have access to secrets — SPAs run in the browser, which is an untrusted environment. If you use a single registration, the SPA's client ID is the same as the API's, and any secret configured on that registration is associated with the same identity. With two registrations, the SPA has no secrets at all — it uses Authorization Code + PKCE, which requires no secret.
- **Scope control.** With a single registration, the app is both the requester and the provider of permissions. There is no way to restrict which scopes the SPA can request — it can use whatever it wants, because it is the same app. With two registrations, the API exposes scopes (e.g., `access_as_user`), and the UI is explicitly granted only the scopes it needs. This enforces least privilege.
- **Independent lifecycle.** The UI and the API have different deployment cycles, different teams, different security postures. Separate registrations let you rotate API credentials without touching the UI, apply different Conditional Access policies, and track consent independently.
- **OBO breaks.** The On-Behalf-Of flow requires a middle-tier service to exchange a user token for a new token targeting a downstream API. If the SPA and the API share the same client ID,

the OBO exchange becomes the app calling itself, which is not a supported scenario.

i Analogy: Think of it as a building pass. The UI app registration is the visitor badge — it identifies who is at the front door and what they're allowed to ask for. The API app registration is the access list — it defines which rooms exist and who is authorized to enter. Combining them into one badge means the visitor decides their own access level.

"Why can't a single access token work for all my APIs?"

Because an access token in Microsoft Entra is bound to a single audience (`aud` claim). This is by design in the [OAuth 2.0 specification and Microsoft's implementation](#) — each token is a "ticket" valid for one specific destination.

If a single token could target multiple APIs, then:

- A malicious or compromised API (Domain A) could take the token it received and replay it against another API (Domain B) — the user never intended to grant Domain A access to Domain B.
- There would be no way for an API to verify that the token was actually meant for it.

The `aud` claim solves this: Domain B checks that `aud` is `api://domain-b`, and rejects any token meant for someone else.

The user still only signs in once — MSAL silently acquires additional tokens using the cached refresh token. It's multiple tokens, but a single sign-on experience.

"What's the difference between scopes and roles? They both seem to control access."

They control access at different levels:

- **Scopes** (`scp` claim) answer: "What is this **app** allowed to do on behalf of the user?" They are the app's leash.
- **Roles** (`roles` claim) answer: "What is this **caller** (user or service) authorized to be?" They are the caller's badge.

In a delegated flow, both apply. A user with the role `DomainA.Admin` who signs into a UI app that only requested the `read` scope cannot write — the app's leash prevents it, even though the user's badge would allow it.

In an app-only flow (service-to-service), there is no user and no lease — only the service's roles matter.

i Rule of thumb: Use roles for business-level authorization ("who can do what"). Use scopes to limit what client apps can request — especially useful when multiple UI apps with different trust levels access the same API.

"What is `access_as_user` ? Is it a special Microsoft scope?"

No. `access_as_user` is just a naming convention — a custom scope that many teams define on their API app registration under "Expose an API." There is nothing special about the name.

It means: "This app is allowed to call the API as the signed-in user." It's intentionally coarse — it acts as a gatekeeper ("are you allowed to call this API at all?"), while fine-grained authorization is handled by app roles (`DomainA.Reader` , `DomainA.Admin` , etc.).

You could name it `api_access` , `user_impersonation` , `one_punch_scope` , `stranger_scopes` or anything else. Microsoft's own first-party APIs often use `user_impersonation` as their equivalent.

"Where do I assign roles to users — on the UI app or the API app?"

Always on the **API app's enterprise application** (service principal). Never on the UI app.

The UI app is just a client — it requests tokens. The API app is the resource — it defines what roles exist. Users are assigned roles on the resource they are accessing, not on the client they are accessing it from.

In the Entra portal: Microsoft Entra ID → Enterprise applications → find the **API** app → Users and groups → Add user/group → select the role.

"If I use per-domain API registrations, does the user have to sign in multiple times?"

No. The user signs in **once** through the UI app. MSAL.js caches the session and refresh token. When the UI needs a token for a different domain, it calls `acquireTokenSilent` — MSAL uses the cached refresh token to obtain a new access token for that domain without any user interaction.

The user sees one login prompt. Behind the scenes, MSAL may make multiple silent token requests, but these are invisible HTTP calls — no popups, no redirects.

"What's the difference between OBO and client credentials? When do I use each?"

	On-Behalf-Of (OBO)	Client Credentials
User context	Yes — the downstream API sees the original user's identity	No — the downstream API sees only the service's identity
Use case	User-initiated request that needs to call another API while preserving who the user is	Background job, daemon, scheduled task, or any call where no user is involved
Example	User clicks "load report" → UI calls Domain A → Domain A calls Domain B to get supplementary data, still as the user	Nightly sync job copies data from Domain A to Domain B
Token type	Delegated (has <code>scp</code> + <code>roles</code>)	App-only (has <code>roles</code> only)
Requires	The incoming user token + the service's own credentials	Only the service's own credentials

i Rule of thumb: If the action was triggered by a user and the downstream API needs to know which user, use OBO. If no user is involved or the downstream API doesn't care who the user is, use client credentials.

"Why does KrakenD validate the token if the backend also validates it?"

Defense in depth. KrakenD acts as the first line — it rejects expired, malformed, or unsigned tokens before they ever reach the backend. This protects backends from processing invalid requests, reduces attack surface, and offloads common validation logic.

Backends still validate because:

- They perform **fine-grained authorization** (e.g., checking specific role combinations, business rules based on claims).

- They verify the `aud` claim matches their own API — KrakenD routes to many backends, but each backend should only accept tokens meant for it.
- In Strategy A, if the backend needs to perform OBO, it needs the original validated token.

Think of it as airport security (KrakenD) vs. boarding gate check (backend). Both check your boarding pass, but for different reasons.

"Can a managed identity use app roles like a regular service principal?"

Yes. A managed identity (system-assigned or user-assigned) creates a service principal in your tenant. That service principal can be assigned app roles on any API's enterprise application, exactly like any other service principal.

The only difference is how it authenticates — instead of client ID + secret, it uses the Azure Instance Metadata Service (IMDS) to obtain tokens automatically. The resulting token contains the `roles` claim just like any client credentials token.

Assign roles via: Enterprise applications → find the **API** app → Users and groups → Add user/group → search for the managed identity's name → select the role.

"What happens if I define an app role with 'Allowed member types: Users/Groups' only, but a service needs it?"

The service principal will not be able to receive that role. The role will not appear under "Application permissions" when configuring API permissions for the service's app registration.

If both users and services need the same role, set "Allowed member types" to **Both (Users/Groups + Applications)**. This makes the role assignable to users (via enterprise app → Users and groups) and to service principals (via API Permissions → Application → admin consent).